Project – Microservices Milestone – Part 1

Design, Structure, Initial Implementation (10% of the final valuation)

Project: Microservices Milestone – Part 1

Design, Structure, and Initial Implementation

Value: 10% of final grade

Due Date: April 24, 2026

1. Objectives

- **Concepts:** Explain microservices and their primary use cases.
- **Migration:** Decompose a DDD Modular Monolith into a microservices architecture.
- **Infrastructure:** Implement an API Gateway and containerize the entire system.
- **Documentation:** Implement **Swagger/OpenAPI** for every microservice.
- **Development:** Build RESTful services using **Spring Boot** and **H2/PostgreSQL**.
- **Orchestration:** Implement a core data aggregator/orchestrator using **HATEOAS**. This service must include **Ports** for communicating with other services and **ACL (Anti-Corruption Layers)** for implementing those ports.

2. Architecture Overview

You must implement a total of **5 microservices**:

1. **1 API Gateway**
2. **1 Core Orchestrator / Data Aggregator** (Requires HATEOAS, Ports, and ACL).
3. **3 Supporting (Low-Level) Services** (Independent; HATEOAS not required).

Containerization Requirements

The solution must be Dockerized using **9 containers**:

- **5 Containers** for the microservices.
- **4 Containers** for PostgreSQL instances (one for each microservice, excluding the gateway).

3. Implementation Roadmap

Step 1: Design

- Develop **C4 Level 1 (System Context)** and **C4 Level 2 (Container)** diagrams using **C4-PlantUML**.

Step 2: Scaffolding (refer to Assignment 6 and the videos provided on LEA for guidance).

- Generate the skeleton architecture using the **Spring CLI** and set up a **multi-project Gradle build**

Step 3: Development & Documentation (refer to Assignment 7 and 8 plus the videos provided on LEA for guidance).

- **Migration:** Move logic from your DDD monolith **(refer to Assignment 7 as an example).**
- **Database:** Configure **H2 databases** for local testing within IntelliJ.
- **Swagger/OpenAPI:** Integrate springdoc-openapi-ui in **all five services**. Ensure the Swagger UI is accessible for each service to visualize and test endpoints.
- **Error Handling:** Each service must implement at least **two domain-specific** and **two application-specific** exceptions.
- **Presentation Layer:** Implement success paths in standard Controllers and failure paths using **Global** and **Service Exception Handlers** (@ControllerAdvice).
- **Dockerization:** Configure all 9 containers (5 services, 4 PostgreSQL) using **Docker Compose**.

4. Testing Requirements

(refer to "*postman_collection_vrms_microservices*" provided on "*https://github.com/MortezaGitPro/vrms-microservices*" for guidance).

- **Integration Testing:** All services (3 supporting + 1 orchestrator) must be tested via **Postman**. You must demonstrate positive (success) and negative (failure) paths for all HTTP methods (**GET, POST, PUT, DELETE**).
- **API Documentation:** Verify that the **Swagger UI** correctly lists all endpoints, schemas, and possible HTTP response codes for every service.
- **Validation:** Ensure all exception paths return the **correct HTTP status codes**.

5. Grading (Mandatory Attendance)

You will present your work to in-person for grading. **Absence without a college-approved excuse results in a grade of 0.**

During the session, you must:

1. Explain the mapping between your **C4 diagrams/DDD model** and your code.
2. Explain your project code, the migration steps, and the differences between your modular monolith and microservices architecture.
3. Demonstrate that the multi-project build works in both **IntelliJ (using H2 database)** and **Docker/Docker-Compose (using PostgreSQL)**.
4. Show that your **9 docker containers** are running without issue.
5. Open the **Swagger UI** for each service and demonstrate how it documents your API.
6. Execute **10–12 requests per microservice** via Postman to prove full functionality.

6. Submission Guidelines

Submit the following via Moodle:

1. **GitHub Link:** To your project repository.
2. **Diagrams:** 3 separate PNG files (C4 L1, C4 L2, and updated DDD Domain Model).
3. **Video Demonstration:** A video explaining your code and demonstrating all tests. You must show the **Swagger UI** for each service. You must also run all **postman tests** and confirm that tests return the **correct status codes**. You must also show that your docker 9 containers are running without any issue.

Note: Ensure your local H2 testing is bug-free before proceeding to Docker.